Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «Проектирование программного обеспечения интеллектуальных систем»
За 1 семестр
Тема: «Функции-друзья классов»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.
Проверил:
Монтик Н.С.

Брест 2026

Цель работы: Изучить использование функций-друзей (`friend`) для доступа к закрытым членам класса; изучить особенности перегрузки операторов в пользовательских классах; закрепить навыки проектирования классов.

Ход работы:

Задание: На основе класса Student, реализованного в лабораторной работе №3, реализовать: дружественную функцию, имеющую доступ к закрытым членам класса; второй класс и дружественную функцию, работающую одновременно с двумя различными классами; дружественный класс, имеющий доступ ко всем членам другого класса. Построить UML-диаграмму классов.

Описание реализованных дружественных функций и классов:

Класс Student (предметная область «Студент»; закрытые поля: `course` - курс, `group` - группа, `grades` - динамический массив оценок, `rating` - ссылка на внешнюю переменную) взят из предыдущей работы: конструкторы, деструктор, оператор `=` и метод `TO_SHOW` сохранены, из методов доступа оставлены `TO_GET_Course`, `TO_GET_Sum`, `TO_GET_Alive`. Дружественные функции и класс не являются членами класса, поэтому объявления `friend` могут находиться в любой секции класса, а определения пишутся без `friend` и без `Student::`.

1. Дружественная функция одного класса.

- `void TO_SHOW_PRIVATE(const Student& s)` - напрямую обращается к закрытым полям `course`, `group`, `grades` и `rating` объекта и выводит их;
- `ostream& operator<<(ostream& os, const Student& s)` - перегрузка оператора вывода как дружественной функции. Левым операндом является `ostream`, поэтому оператор нельзя сделать методом Student; возврат `ostream&` позволяет писать цепочки `cout << a << b`.

Без объявления `friend` обращение к закрытому полю из внешней функции не компилируется:

```
void show(const Student& s) {
    cout << s.course << endl;
}
```

Ошибка компилятора:

```
error: 'int Student::course' is private within this context
note: declared private here
```

2. Дружественная функция двух классов.

Создан второй класс Subject («Учебный предмет») с закрытыми полями `name` - название, `passMark` - проходной балл, `index` - номер оценки в массиве студента, `examCount` - число выставленных оценок. Две функции объявлены другом сразу в обоих классах, поэтому имеют доступ к закрытым членам Student и Subject одновременно:

- `bool TO_CHECK_PASS(const Student& s, const Subject& sub)` - читает оценку `s.grades[sub.index]` и сравнивает её с `sub.passMark`; возвращает `true`, если предмет сдан;
- `void TO_EXAM(Student& s, Subject& sub, int mark)` - изменяет закрытые поля обоих классов: записывает оценку в `s.grades[sub.index]` и увеличивает `sub.examCount`.

Так как Student упоминает Subject до его определения, перед классом Student размещено предварительное объявление `class Subject;`.

3. Дружественный класс.

Класс Deanery («Деканат») объявлен другом Student (`friend class Deanery;`), поэтому все его методы имеют доступ ко всем членам Student:

- `TO_PROMOTE(Student& s)` - увеличивает закрытое поле `course` (не выше 5);
- `TO_SET_Rating(Student& s, int r)` - записывает значение в закрытую ссылку `rating`, то есть изменяет внешнюю переменную;
- `TO_RESET_Grades(Student& s)` - обнуляет закрытый динамический массив оценок;

- `TO_REPORT(const Student& s) const` - читает все закрытые поля и выводит отчёт.

Дружба односторонняя: Deanery получает доступ к Student, но не наоборот, поэтому закрытое поле `processed` класса Deanery для Student недоступно. Дружба также не наследуется и не является транзитивной.

4. Демонстрация работы.

В функции `main()` создаётся студент `s`, привязанный к переменной `groupRating`, и три предмета: Математика (проходной 6, оценка №1), Физика (проходной 8, оценка №4), Химия (проходной 8, оценка №3). Сначала вызываются функции из пункта 1, затем проверка сдачи и пересдача химии функциями из пункта 2 (оценка 7 меняется на 9, химия становится сданной), затем деканат переводит студента на 3 курс, устанавливает рейтинг 95 (значение `groupRating` меняется) и обнуляет оценки.

Код программы:

```
#include <iostream>
#include <string>
#include <cstdlib>
#include <clocale>
using namespace std;

class Subject;

class Student
{
public:
    static const int SIZE = 5;

private:
    int course;
    int group;
    int *grades;
    int &rating;
    static int alive;
    static int defaultRating;

public:
    Student() : rating(defaultRating)
    {
        course = 1;
        group = 0;
        grades = new int[SIZE];
```

```
    for (int i = 0; i < SIZE; i++)
    {
        grades[i] = 0;
    }
    alive++;
}

Student(int c, int g, const int arr[], int &r) : rating(r)
{
    course = c;
    group = g;
    grades = new int[SIZE];
    for (int i = 0; i < SIZE; i++)
    {
        grades[i] = arr[i];
    }
    alive++;
}

Student(const Student &other) : rating(other.rating)
{
    course = other.course;
    group = other.group;
    grades = new int[SIZE];
    for (int i = 0; i < SIZE; i++)
    {
        grades[i] = other.grades[i];
    }
    alive++;
}

~Student()
{
    delete[] grades;
    alive--;
}

Student &operator=(const Student &other)
{
    if (this != &other)
    {
        delete[] grades;
        course = other.course;
        group = other.group;
        grades = new int[SIZE];
        for (int i = 0; i < SIZE; i++)
        {
            grades[i] = other.grades[i];
        }
    }
    return *this;
}
```

```
    void TO_SHOW() const
    {
        cout << "Курс: " << course;
        cout << " | Группа: " << group;
        cout << " | Оценки: ";
        for (int i = 0; i < SIZE; i++)
        {
            cout << grades[i] << " ";
        }
        cout << "| Рейтинг: " << rating << endl;
    }

    int TO_GET_Course() const { return course; }

    int TO_GET_Sum() const
    {
        int sum = 0;
        for (int i = 0; i < SIZE; i++)
        {
            sum += grades[i];
        }
        return sum;
    }

    static int TO_GET_Alive() { return alive; }

    friend void TO_SHOW_PRIVATE(const Student &s);

    friend ostream &operator<<(ostream &os, const Student &s);

    friend bool TO_CHECK_PASS(const Student &s,
                              const Subject &sub);
    friend void TO_EXAM(Student &s, Subject &sub, int mark);

    friend class Deanery;
};

int Student::alive = 0;
int Student::defaultRating = 0;

class Subject
{
private:
    string name;
    int passMark;
    int index;
    int examCount;

public:
    Subject(const string &n, int p, int i)
        : name(n), passMark(p), index(i), examCount(0) {}

    int TO_GET_ExamCount() const { return examCount; }
```

```
    friend bool TO_CHECK_PASS(const Student &s,
                              const Subject &sub);
    friend void TO_EXAM(Student &s, Subject &sub, int mark);
};

void TO_SHOW_PRIVATE(const Student &s)
{
    cout << "[friend] course = " << s.course
         << ", group = " << s.group << endl;
    cout << "[friend] grades = [";
    for (int i = 0; i < Student::SIZE; i++)
    {
        cout << s.grades[i] << (i < Student::SIZE - 1 ? ", " :
"");
    }
    cout << "], rating = " << s.rating << endl;
}

ostream &operator<<(ostream &os, const Student &s)
{
    os << "Student{курс=" << s.course << ", группа=" << s.group
       << ", оценки=[";
    for (int i = 0; i < Student::SIZE; i++)
    {
        os << s.grades[i] << (i < Student::SIZE - 1 ? " " : "");
    }
    os << "], рейтинг=" << s.rating << "}";
    return os;
}

bool TO_CHECK_PASS(const Student &s, const Subject &sub)
{
    int mark = s.grades[sub.index];
    bool ok = mark >= sub.passMark;
    cout << "  " << sub.name << " (оценка №" << sub.index + 1
         << "): " << mark << ", проходной " << sub.passMark
         << " -> " << (ok ? "сдан" : "не сдан") << endl;
    return ok;
}

void TO_EXAM(Student &s, Subject &sub, int mark)
{
    s.grades[sub.index] = mark;
    sub.examCount++;
    cout << "  Экзамен «" << sub.name << "»: оценка " << mark
         << " (экзаменов: " << sub.examCount << ")" << endl;
}

class Deanery
{
private:
    int processed;
```

```
public:
    Deanery() : processed(0) {}

    void TO_PROMOTE(Student &s)
    {
        if (s.course < 5)
        {
            s.course++;
        }
        processed++;
        cout << "  Перевод: теперь курс " << s.course << endl;
    }

    void TO_SET_Rating(Student &s, int r)
    {
        s.rating = r;
        processed++;
        cout << "  Рейтинг установлен: " << r << endl;
    }

    void TO_RESET_Grades(Student &s)
    {
        for (int i = 0; i < Student::SIZE; i++)
        {
            s.grades[i] = 0;
        }
        processed++;
        cout << "  Оценки обнулены" << endl;
    }

    void TO_REPORT(const Student &s) const
    {
        cout << "  Отчёт: курс " << s.course << ", группа "
             << s.group << ", рейтинг " << s.rating
             << ", сумма оценок " << s.TO_GET_Sum() << endl;
    }

    int TO_GET_Processed() const { return processed; }
};

int main()
{
    system("chcp 65001");
    setlocale(LC_ALL, "ru_RU.UTF-8");

    int g[Student::SIZE] = {8, 9, 7, 10, 9};
    int groupRating = 70;
    Student s(2, 29, g, groupRating);

    cout << "=== 1. Дружественная функция ===" << endl;
    cout << "Открытый метод TO_SHOW: ";
    s.TO_SHOW();
```

```
    TO_SHOW_PRIVATE(s);
    cout << "operator<<: " << s << endl;

    cout << endl
         << "=== 2. Дружественная функция двух классов ==="
         << endl;
    Subject math("Математика", 6, 0);
    Subject phys("Физика", 8, 3);
    Subject chem("Химия", 8, 2);

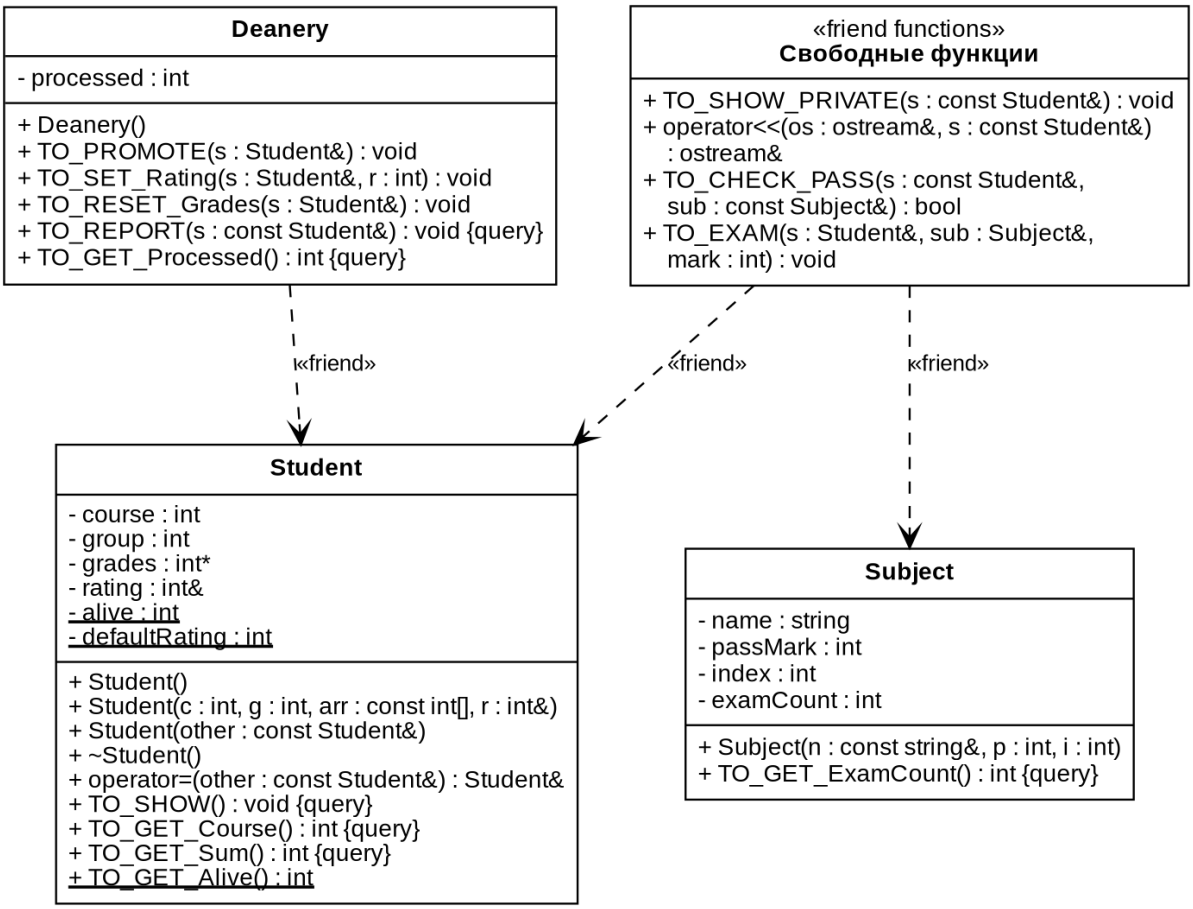
    cout << "Проверка сдачи:" << endl;
    TO_CHECK_PASS(s, math);
    TO_CHECK_PASS(s, phys);
    TO_CHECK_PASS(s, chem);

    cout << "Пересдача химии:" << endl;
    TO_EXAM(s, chem, 9);
    TO_CHECK_PASS(s, chem);
    cout << "s: " << s << endl;
    cout << "Экзаменов по химии: "
         << chem.TO_GET_ExamCount() << endl;

    cout << endl
         << "=== 3. Дружественный класс ===" << endl;
    Deanery dean;
    cout << "До: " << s << endl;
    dean.TO_PROMOTE(s);
    dean.TO_SET_Rating(s, 95);
    cout << "groupRating = " << groupRating << endl;
    dean.TO_REPORT(s);
    dean.TO_RESET_Grades(s);
    cout << "После: " << s << endl;
    cout << "Операций деканата: "
         << dean.TO_GET_Processed() << endl;

    return 0;
}
```

UML-диаграмма классов:



Знак – обозначает закрытые, + - открытые члены; статические члены подчёркнуты. Зависимости со стереотипом «friend» направлены от друга к классу, предоставившему доступ: класс Deanery и набор свободных функций получают доступ к Student, а свободные функции - ещё и к Subject.

Результат:

1. Дружественная функция одного класса:

```
=== 1. Дружественная функция ===
Открытый метод TO_SHOW: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9 | Рейтинг: 70
[friend] course = 2, group = 29
[friend] grades = [8, 9, 7, 10, 9], rating = 70
operator<<: Student{курс=2, группа=29, оценки=[8 9 7 10 9], рейтинг=70}
```

2. Дружественная функция двух классов:

```
=== 2. Дружественная функция двух классов ===
Проверка сдачи:
  Математика (оценка №1): 8, проходной 6 -> сдан
  Физика (оценка №4): 10, проходной 8 -> сдан
  Химия (оценка №3): 7, проходной 8 -> не сдан
Пересдача химии:
  Экзамен «Химия»: оценка 9 (экзаменов: 1)
  Химия (оценка №3): 9, проходной 8 -> сдан
s: Student{курс=2, группа=29, оценки=[8 9 9 10 9], рейтинг=70}
Экзаменов по химии: 1
```

3. Дружественный класс:

```
=== 3. Дружественный класс ===
До: Student{курс=2, группа=29, оценки=[8 9 9 10 9], рейтинг=70}
  Перевод: теперь курс 3
  Рейтинг установлен: 95
groupRating = 95
  Отчёт: курс 3, группа 29, рейтинг 95, сумма оценок 45
  Оценки обнулены
После: Student{курс=3, группа=29, оценки=[0 0 0 0 0], рейтинг=95}
Операций деканата: 3
```

Вывод: Изучили использование функций-друзей и реализовали на основе класса Student дружественную функцию одного класса (включая перегрузку оператора <<), дружественные функции, работающие одновременно с классами Student и Subject, и дружественный класс Deanery. Показали, что дружественные функции и классы получают прямой доступ к закрытым членам, включая динамический массив и ссылочное поле, и могут не только читать, но и изменять их. Выяснили, что дружба предоставляется самим классом, является односторонней и нарушает инкапсуляцию, поэтому применять её следует ограниченно. Построили UML-диаграмму классов со связями «friend».